Session 1

Introduction to Software Engineering Tools

Course:	Software Engineering Tools Lab (IT-B, 2026-27)
Project:	AI Interview Preparation Platform
Team:	Hirdesh (Lead), Mahek Bhatia, Janhavi Srivastava
Deliverable:	GitHub Repository Setup
Industry Mapping:	Collaborative Development

1. Session Objectives

This session introduces the foundational tools and workflows that every software engineer uses in industry. By the end of this session, students will be able to:

- Understand the purpose and value of version control in collaborative software development.
- Install and configure Git on their local machine.
- Create and manage a GitHub repository for a team project.
- Perform core Git operations: init, add, commit, push, pull, branch, and merge.
- Describe the phases of the Software Development Life Cycle (SDLC).
- Form project teams and select a real-world problem domain for the semester project.

2. Introduction to Git and GitHub

2.1 What is Version Control?

Version control is a system that records changes to files over time so that specific versions can be recalled later. It allows multiple developers to work on the same codebase without overwriting each other's changes, provides a complete history of modifications, and enables easy rollback to a previous state.

2.2 Why Git?

Git is the industry-standard distributed version control system created by Linus Torvalds in 2005. Key advantages include:

- Distributed architecture - every developer has a full copy of the repository history.
- Branching and merging - lightweight branches make parallel development seamless.
- Speed - nearly all operations are performed locally, making Git extremely fast.

- Data integrity - every file and commit is checksummed using SHA-1 hashing.
- Open source - free to use with a massive ecosystem of tools and integrations.

2.3 Git vs GitHub

Git is the version control tool that runs locally on a developer's machine. GitHub is a cloud-hosted platform that provides remote Git repository hosting along with collaboration features such as pull requests, issue tracking, project boards, CI/CD via GitHub Actions, and code reviews.

2.4 Installing Git

Download from https://git-scm.com and follow the installer. Verify installation:

```
git --version
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

3. Repository Creation & Version Control Basics

3.1 Creating a Repository on GitHub

1. Sign in to GitHub and click "New repository".
2. Enter a descriptive repository name (e.g., 2026-27_IT-B_202401100600089).
3. Add a README, choose a license (MIT), and create the repository.
4. Clone the repository to your local machine:

```
git clone https://github.com/<username>/<repo-name>.git
```

3.2 Core Git Workflow

The basic daily workflow in Git follows these steps:

1. Pull latest changes: git pull origin main
2. Create / switch to a feature branch: git checkout -b feature/login-page
3. Make changes and stage them: git add .
4. Commit with a meaningful message: git commit -m "Add login page UI"
5. Push the branch: git push origin feature/login-page
6. Open a Pull Request on GitHub for code review and merge.

3.3 Essential Git Commands Reference

```
git init              # Initialise a new repository
git status            # Check working tree status
git add <file>        # Stage changes
```

```
git commit -m 'msg'   # Commit staged changes
git log --oneline     # View commit history
git branch <name>     # Create a branch
git checkout <name>   # Switch branches
git merge <branch>    # Merge a branch into current
git pull              # Fetch + merge from remote
git push              # Push commits to remote
```

4. Software Project Lifecycle in Industry

4.1 Software Development Life Cycle (SDLC)

The SDLC is a structured framework that defines the phases involved in building software systems. Understanding the SDLC helps teams plan, execute, and deliver high-quality software products.

- **Requirement Analysis:** Gather and document what the system must do. Produce SRS documents.
- **System Design:** Define architecture, data models, UI wireframes, and technology stack.
- **Implementation:** Write code, integrate components, and follow coding standards.
- **Testing:** Verify functionality via unit, integration, system, and acceptance testing.
- **Deployment:** Release the product to staging/production environments using CI/CD pipelines.
- **Maintenance:** Monitor, fix bugs, add features, and optimise based on user feedback.

4.2 Agile Methodology Overview

Most modern software teams follow Agile practices. Agile emphasises iterative development, continuous feedback, and cross-functional collaboration. Key Agile ceremonies include Sprint Planning, Daily Stand-ups, Sprint Reviews, and Retrospectives. Tools like Jira, Trello, and GitHub Projects are commonly used to manage Agile workflows.

5. Team Formation & Project Domain Selection

5.1 Team Composition

For this semester, each team consists of 2-4 members with clearly defined roles:

- Team Lead - coordinates tasks, manages the repository, and ensures deadlines are met.
- Developers - responsible for front-end, back-end, and/or AI/ML modules.
- All members participate in code reviews, documentation, and testing.

5.2 Our Team

- **Hirdesh:** Team Lead - Repository management, architecture, backend

- **Mahek Bhatia:** Frontend development, UI/UX design
- **Janhavi Srivastava:** AI/ML integration, testing

5.3 Selected Project Domain

Our team has selected the domain of AI-Powered Interview Preparation. The project, "AI Interview Preparation Platform", aims to simulate technical and behavioral interviews using AI, provide instant feedback on responses, and analyse resumes for ATS compatibility.

Technology Stack:

- Frontend: Next.js, Tailwind CSS
- Backend: FastAPI / NestJS
- AI/ML: Gemini API, OpenAI API
- Database: PostgreSQL, Redis

6. Deliverable: GitHub Repository Setup

The following repository has been created and configured for the semester project:

Repository URL: https://github.com/hirdeshds/2026-27_IT-B_202401100600089

Repository structure:

```
2026-27_IT-B_202401100600089/
+-- src/             # Application source code
+-- test/            # Test suites
+-- experiment/      # Experimental notebooks and scripts
+-- session/         # Session documents and deliverables
+-- README.md        # Project documentation
+-- LICENSE          # MIT License
```

Completed setup tasks:

- Repository created on GitHub with README and MIT License.
- Folder structure initialised (src/, test/, experiment/, session/).
- Team members added as collaborators with appropriate access permissions.
- Branch protection rules configured for the main branch.

7. Industry Mapping: Collaborative Development

This session maps directly to real-world collaborative development practices used across the software industry:

- Companies like Google, Microsoft, and Meta use Git-based workflows for all code management.

- GitHub / GitLab / Bitbucket serve as the central collaboration hubs for development teams.
- Code reviews via pull requests are a standard quality assurance practice.
- CI/CD pipelines triggered by Git events automate testing and deployment.
- Teams follow branching strategies (Git Flow, Trunk-Based Development) to manage releases.
- Every professional developer is expected to be proficient in Git from day one.

8. Session Summary

In this session, we covered the fundamentals of version control using Git and GitHub, understood the software project lifecycle phases used in industry, formed our project team, and selected the AI Interview Preparation Platform as our semester project domain. The GitHub repository has been set up with an organised folder structure, a comprehensive README, and appropriate collaboration settings.

Next Session: Requirements Engineering - SRS Document and Use Case Modelling.